

# KumoLang: A Formal Verification Framework for Cheesecake Synthesis<sup>‡§</sup>

Sean L.      Thomas W.      Eric Y.      Jack Z.      Hachimi R.

June 11, 2026

## Abstract

We present **KumoLang**, a purely imperative specification language and verification framework that formalizes culinary execution procedures through linear state transitions and structural correctness-by-construction, and an proof-of-concept verification engine in Rust (<https://github.com/makabaka1880/kumolang>). Unlike traditional stateful architectures that rely on decoupled static analysis or complex reference counting, the operational semantics of KumoLang inherently guarantee a well-formed, acyclic dependency trace during runtime evaluation. We formalize the core syntax and execution state through a list-monad architecture: raw components are lifted into isolated container types via a monadic `prep` constructor, combined symmetrically via an endomorphic `pour` operator, and transitioned across distinct nominal domains using a type-lifting `pour-out` functor. Because the structural rules of the calculus strictly forbid invalid resource configurations, ill-formed physical configurations—such as domain cross-contamination or unbound variable references—are rendered completely unrepresentable. This tight coupling of language semantics and safety invariants allows the framework to enforce absolute domain boundaries natively, ensuring the deterministic, verified synthesis of a target KumoKumo Basque pastry specification with zero compilation or runtime verification overhead.

---

<sup>\*</sup>This is a joke paper. However, KumoLang is a language that actually works, and if one does not mind expressing recipes in a Lisp-like syntax, it is entirely suitable for daily cheesecake synthesis workflows.

<sup>†</sup>Portions of this manuscript were generated with the assistance of AI tools. The language design, operational semantics, and formalization were developed by the authors. While the presentation is intentionally comedic, the underlying formal methods are genuine.

<sup>‡</sup>The authors thank Kumo for an excellent APCSA course.

<sup>§</sup>The authors thank KumoKumo for producing the Basque cheesecakes that inspired this work.

# 1 Introduction

In his foundational work *Tractatus Logico-Philosophicus*, Ludwig Wittgenstein famously asserted, “The world is everything that is the case. The world is the totality of facts, not of things” [1]. If we extend this analytical framework to the physical bounds of the culinary arts, we encounter a profound systemic failure in modern procedural tracking. The modern kitchen is not a loose collection of macroscopic things (e.g., cheese, eggs, vessels); it is a rigorous totality of state-based facts, thermodynamic boundaries, and irreversible substructural transformations.

Traditionally, the formal verification of physical procedures has been severely neglected by the programming language theory (PLT) community. This oversight has left the domain of recipe analysis primarily to coarse natural language processing (NLP) heuristics and machine learning dependency graphs designed for general commonsense reasoning [2]. These stochastic methods are fundamentally inadequate. They treat a physical sequence as a relaxed recommendation rather than what it truly is: a stateful, resource-bounded program. In a physical environment, resources are consumed linearly, and state transformations are destructive and non-monotonic. An egg, once emulsified into a dairy matrix, cannot be recovered via structural weakening or contraction.

To rectify this foundational gap, we introduce **KumoLang**, a purely imperative specification language and verification framework engineered to formalize the synthesis of complex multi-component pastries—specifically optimized for a localized compilation target of a KumoKumo Basque cheesecake. Unlike traditional verification platforms that require upfront static linting passes to check resource safety, the structural semantics of KumoLang inherently guarantee a well-formed directed acyclic graph (DAG) of physical dependencies directly during linear statement evaluation. By mapping execution to a strict sequential list-monad algebra, the small-step operational evaluation of an un-nested program naturally achieves a state of *Correctness-by-Construction*: raw mixtures are instantiated as isolated, generative nominal types, which are subsequently manipulated as linear container assets. The resulting abstract semantic trace renders invalid physical states completely unrepresentable, ensuring the deterministic and verified synthesis of the target pastry specification with zero runtime overhead.

The remainder of this paper is organized as follows. Section 2 presents the abstract syntax and small-step operational semantics of KumoLang, formalizing its core monadic operators and the structural invariants they enforce. Section 3 introduces the Kumo Verification Engine and walks through a complete trace evaluation for a canonical cheesecake synthesis program. Section 4 concludes, and Section 5 outlines directions for future work.

## 2 Syntax and Abstract Architecture

We now formalize the core calculus that underpins the guarantees outlined above. The language utilizes an S-Expression syntax restricted to variable identifiers for operational arguments, structurally enforcing a clear line of resource custody. We begin with the formal grammar, then define the semantic domains over which evaluation operates, and finally present the small-step operational semantics that constitute the heart of the verification framework.

### 2.1 Formal Grammar

Let  $x, y, z \in \text{Var}$  range over variable identifiers tracking active mixtures or container states. A KumoLang program  $P$  consists of a sequentially composed chain of statements  $s$ , defined inductively by the following abstract syntax:

|                                                    |                                                     |
|----------------------------------------------------|-----------------------------------------------------|
| $P ::= s$                                          | (Instruction)                                       |
| $\quad   \quad s ; P$                              | (Sequence)                                          |
| $s ::= \text{skip}$                                | (Stasis)                                            |
| $\quad   \quad x \leftarrow \text{new-mixture}()$  | (Domain allocation)                                 |
| $\quad   \quad y \leftarrow \text{prep}(x)$        | (Lift mixture $x$ to container $y$ )                |
| $\quad   \quad y \leftarrow \text{pour}(y, z)$     | (Merge container $z$ into container $y$ )           |
| $\quad   \quad z \leftarrow \text{pour-out}(y, x)$ | (Type casting: lift container $y$ into domain $x$ ) |

### 2.2 Semantic Domains

Having specified the syntactic forms, we now define the semantic domains over which program evaluation operates. Unlike conventional general-purpose programming languages designed for value transformation or numerical computation, KumoLang is a domain-specific specification language. Its operational semantics do not model a traditional abstract machine; rather, they formally characterize the evolution of valid structural relationships and resource tracking. An execution trace represents a sequence of logical validations proving that a physical process is structurally coherent.

We define the semantic domains to track nominal mixture scopes and container configurations. Let  $\mathcal{M}$  be a countable set of generative mixture domain tokens, and  $\text{Var}$  be the set of variable identifiers.

|                                                                                     |                                  |
|-------------------------------------------------------------------------------------|----------------------------------|
| $m \in \mathcal{M}$                                                                 | (Nominal Mixture Identifiers)    |
| $L \in \mathcal{L}_m = \text{List}(m)$                                              | (Structural History List Domain) |
| $\sigma \in \text{Store} = \text{Var} \rightarrow (\mathcal{M} \cup \mathcal{L}_m)$ | (Structural Environment State)   |

When a variable identifier is resolved in the environment store  $\sigma$ , it maps to one of two structural categories:

- **Mixture Labels** ( $m \in \mathcal{M}$ ): Act as a strict type-level boundary. Objects belonging to separate mixture universes cannot cross-contaminate.
- **Container Configurations** ( $L \in \mathcal{L}_m$ ): Represent the physical state of a vessel, formalized as a list tracking the chronological history log of components poured into it.

### 2.3 Small-Step Operational Semantics

With the store model in place, we now define the evaluation relation that governs program execution. A state configuration is defined as a pair  $\langle P, \sigma \rangle$ , where  $P$  is the remaining program sequence and  $\sigma$  is the current environmental store. The structural evaluation relation  $\langle P, \sigma \rangle \rightarrow \langle P', \sigma' \rangle$  defines a single step of structural validation. We present the inference rules for each statement form below.

$$\begin{array}{c}
\text{Stasis} \frac{}{\langle \text{skip} ; P, \sigma \rangle \rightarrow \langle P, \sigma \rangle} \\
\\
\text{Domain-Alloc} \frac{m \notin \text{img}(\sigma)}{\langle x \leftarrow \text{new-mixture}() ; P, \sigma \rangle \rightarrow \langle P, \sigma[x \leftarrow m] \rangle} \\
\\
\text{Ing-Prep} \frac{\sigma(x) = m}{\langle y \leftarrow \text{prep}(x) ; P, \sigma \rangle \rightarrow \langle P, \sigma[y \leftarrow [m]] \rangle} \\
\\
\text{Cont-Pour} \frac{\sigma(y) = L_1 \in \mathcal{L}_m \quad \sigma(z) = L_2 \in \mathcal{L}_m}{\langle y \leftarrow \text{pour}(y, z) ; P, \sigma \rangle \rightarrow \langle P, \sigma[y \leftarrow L_1 \uplus L_2] \rangle} \\
\\
\text{Pour-Out} \frac{\sigma(y) = L_1 \in \mathcal{L}_m \quad \sigma(x) = m_{\text{target}}}{\langle z \leftarrow \text{pour-out}(y, x) ; P, \sigma \rangle \rightarrow \langle P, \sigma[y \leftarrow [], z \leftarrow [m_{\text{target}}]] \rangle}
\end{array}$$

## 3 The Kumo Verification Engine

The operational semantics of Section 2 define, in principle, what constitutes a valid culinary execution. In practice, however, verifying that a given program actually produces the desired pastry requires a concrete evaluation engine. The Kumo Verification Engine realizes this by computing small-step reduction sequences from an initial program-store configuration and extracting the resulting dependency trace as a certifiable artifact. We now illustrate this process end-to-end.

### 3.1 Trace Evaluation

We demonstrate the evaluation of a simple KumoLang program that synthesizes a basic cheesecake specification. Consider the following program, which allocates two mixture domains (a base mixture for ingredients and a baking mixture for the final product), prepares two ingredients from the base domain, combines them via a pour, and finally transitions the combined container into the baking domain.

```

BaseMix ← new-mixture() ;
BakeMix ← new-mixture() ;
ccheese ← prep(BaseMix) ;
csugar ← prep(BaseMix) ;
cbowl ← pour(ccheese, csugar) ;
ccake ← pour-out(cbowl, BakeMix) ;
skip

```

We evaluate the operational semantics on this program by applying the inference rules of Section 2.3. Let the initial store be the empty environment:

$$\sigma_0 = \emptyset.$$

The execution proceeds through a sequence of small-step reductions.

|                                                                           |              |
|---------------------------------------------------------------------------|--------------|
| $\langle P_0, \sigma_0 \rangle \rightarrow \langle P_1, \sigma_1 \rangle$ | DOMAIN-ALLOC |
| $\rightarrow \langle P_2, \sigma_2 \rangle$                               | DOMAIN-ALLOC |
| $\rightarrow \langle P_3, \sigma_3 \rangle$                               | ING-PREP     |
| $\rightarrow \langle P_4, \sigma_4 \rangle$                               | ING-PREP     |
| $\rightarrow \langle P_5, \sigma_5 \rangle$                               | CONT-POUR    |
| $\rightarrow \langle P_6, \sigma_6 \rangle$                               | POUR-OUT     |
| $\rightarrow \langle \text{skip}, \sigma_6 \rangle$                       | STASIS       |

where the intermediate stores, obtained by applying the corresponding semantic rule at each step, are given by

$$\sigma_1 = \{\text{BaseMix} \mapsto m_1\},$$

$$\sigma_2 = \{\text{BaseMix} \mapsto m_1, \text{BakeMix} \mapsto m_2\},$$

$$\sigma_3 = \sigma_2[c_{\text{cheese}} \mapsto [m_1]],$$

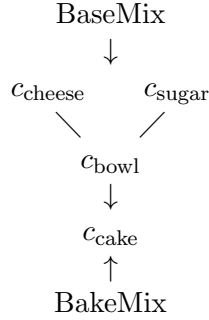
$$\sigma_4 = \sigma_3[c_{\text{sugar}} \mapsto [m_1]],$$

$$\sigma_5 = \sigma_4[c_{\text{bowl}} \mapsto [m_1, m_1]],$$

$$\sigma_6 = \sigma_5[c_{\text{bowl}} \mapsto [], c_{\text{cake}} \mapsto [m_2]].$$

The final configuration certifies the successful construction of a cheese-cake artifact within the baking domain  $m_2$ : variable  $c_{\text{cake}}$  is bound to the singleton history list  $[m_2]$ , confirming that the combined contents of the bowl have been correctly transitioned into the target baking domain. Meanwhile,  $c_{\text{bowl}}$  has been drained to the empty list  $[],$  reflecting the physical reality that the bowl is now empty—its contents having been irreversibly transferred.

This execution trace can be interpreted as a dependency graph describing the provenance of the synthesized culinary artifact. Each node represents a variable binding event, and each edge captures a resource-flow dependency induced by the semantic rules. The resulting graph for our example program is shown below.



The Kumo Verification Engine extracts this graph automatically from the operational trace. Provenance queries may subsequently be answered through graph traversal, while culinary genealogy analysis may be performed through subgraph matching and graph-isomorphism techniques. The algorithm of trace extraction is relatively trivial and will not be given in detail here.

### 3.2 Implementation

The trace extraction and graph construction procedures described above have been realized in a prototype implementation. A Rust implementation of the Kumo Verification Engine is available at <https://github.com/makabaka1880/kumolang>. The engine accepts programs in a concrete S-expression syntax (detailed in the Appendix), computes the full small-step reduction sequence, and outputs the corresponding provenance graph.

## 4 Conclusion

We have presented KumoLang, a purely imperative specification language whose operational semantics provide correctness-by-construction guarantees for physical culinary procedures. By encoding resource manipulation as a list-monad algebra over nominal mixture domains, the framework renders invalid configurations—cross-contamination, unbound references, and structural cycles—completely unrepresentable. The Kumo Verification Engine demonstrates that these guarantees can be realized in practice, producing certifiable provenance graphs from small-step reduction traces with zero run-time verification overhead. While the current framework is scoped to linear, non-recursive pastry synthesis, the underlying semantic architecture is general and invites extension to more expressive culinary settings.

## 5 Future Work

Several extensions remain unexplored. First, we intend to extend KumoLang with support for recursive pastry generation and higher-order baking combinators. Second, we plan to investigate distributed compilation across multiple ovens. Finally, we hope to formalize a dependent type system capable of statically ruling out crust formation.

## 6 Related Work

Up to our knowledge, KumoLang is the first formal verification framework designed specifically for culinary procedure synthesis, especially in the domain of cheese basque cooking.

## 7 Appendix

### 7.1 Concrete Syntax for Implementation

```
<stmt>      ::= <skip_stmt>
               | ( new-mixture <id> )
               | ( prep      <id> <id> )
               | ( pour      <id> <id> <id> )
               | ( pour-out  <id> <id> <id> )

<skip_stmt> ::= skip
               | ( skip )

<id>         matches any valid identifier parsed by ‘parse_ident’
```

## 7.2 Proofs

**Theorem 1** (Basque Preservation). *If a program  $P$  evaluates to a verified Basque cheesecake artifact, then no execution trace of  $P$  contains a crust construction event.*

*Proof.* By inspection of the operational semantics. No rule introduces a crust.  $\square$

## References

- [1] L. Wittgenstein, *Tractatus Logico-Philosophicus*, Routledge & Kegan Paul, 1922.
- [2] A. Bikakis, A. Diallo, L. Dickens, A. Hunter, and R. Miller, *A Graphical Formalism for Commonsense Reasoning with Recipes*, arXiv preprint arXiv:2306.09042, 2023.